

HW3 Part B: Three Chunking Techniques, Retrieval-Only

DATA-260 · Karthik Pragada · DOMAIN_ID 0 (Campus Course Catalogue and Enrolment)

0. Shared setup

Corpus: 16 public SJSU pages, 207,022 bytes (`corpus/hw03/`; see SOURCES.md and CORPUS_MANIFEST.json). **Embedding model:** `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions, local, from HuggingFace), used for all three techniques and for the semantic splitter. **Vector store:** in-memory `SimpleVectorStore`. **Questions:** the five in `questions.yaml` (committed before any retrieval was run). **Top-k:** 5.

Settings used (from `rag_chunking_comparison.py`):

```
EMBED_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
TOP_K = 5
TOKEN_CHUNK_SIZE = 256
TOKEN_CHUNK_OVERLAP = 32
SEMANTIC_BUFFER_SIZE = 1
SEMANTIC_BREAKPOINT_PERCENTILE = 95
SENTENCE_WINDOW_SIZE = 3
```

The retrieval-only function (shared by all three techniques)

Given an index, a query and k, `retrieve_and_analyze` (1) embeds the query and reports its dimension and first 8 values; (2) retrieves the top-k nodes and records the retriever's **store score**; (3) **re-embeds each returned chunk's text explicitly** and computes the cosine similarity to the query vector; (4) records chunk length and a 160-character preview; (5) prints the shapes of the query vector and the stacked document vectors; and (6) times the similarity search only. Every retrieved row is saved to `reports/hw03/raw/` as JSON and CSV.

```
def cosine_sim(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def retrieve_and_analyze(index, embed_model, technique: str, question_id: str, query: str, k: int =
TOP_K, verbose: bool = True):
    query_vec = np.array(embed_model.get_query_embedding(query))

    retriever = index.as_retriever(similarity_top_k=k)
    start = time.perf_counter()
    retrieved = retriever.retrieve(query)
    latency_ms = (time.perf_counter() - start) * 1000

    doc_vecs = []
    rows = []
    for rank, node_with_score in enumerate(retrieved, start=1):
        node = node_with_score.node
        chunk_text = node.get_content()
        doc_vec = np.array(embed_model.get_text_embedding(chunk_text))
        doc_vecs.append(doc_vec)
        sim = cosine_sim(query_vec, doc_vec)
        rows.append({
            "rank": rank,
            "store_score": round(float(node_with_score.score), 6) if node_with_score.score is not None
else None,
            "cosine_sim": round(sim, 6),
```

```

        "chunk_len": len(chunk_text),
        "source_file": node.metadata.get("file_name"),
        "preview": chunk_text[:160].replace("\n", " "),
    })

doc_matrix = np.stack(doc_vecs) if doc_vecs else np.zeros((0, len(query_vec)))

if verbose:
    print(f"\n=== [{technique}] {question_id}: {query.strip()[:80]} ===")
    print(f"query embedding dim: {query_vec.shape[0]}, first 8 values:
{query_vec[:8].round(4).tolist()}")
    print(f"query vector shape: {query_vec.shape}, stacked doc vectors shape: {doc_matrix.shape}")
    print(f"'rank':<5}'store_score':<13}'cosine_sim':<12}'chunk_len':<10}'source_file':
<45}'preview'")
    for r in rows:
        print(f"{r['rank']:<5}{str(r['store_score']):<13}{r['cosine_sim']:<12}{r['chunk_len']:<10}
{str(r['source_file']):<45}{r['preview']}")
    print(f"retrieval latency: {latency_ms:.2f} ms")

return {
    "technique": technique,
    "question_id": question_id,
    "query": query,
    "query_embed_dim": int(query_vec.shape[0]),
    "query_embed_first8": query_vec[:8].round(6).tolist(),
    "latency_ms": round(latency_ms, 3),
    "results": rows,
}

```

Token chunking

1. Chunking

Splits the text every 256 tokens with a 32-token (12.5%) overlap. The size keeps each chunk on roughly one policy or requirement, and the overlap means a fact that straddles a boundary still appears whole in at least one chunk. It ignores meaning: boundaries fall wherever the token count lands.

```

def build_token_index(docs, embed_model):
    splitter = TokenTextSplitter(chunk_size=TOKEN_CHUNK_SIZE, chunk_overlap=TOKEN_CHUNK_OVERLAP)
    nodes = splitter.get_nodes_from_documents(docs)
    index = VectorStoreIndex(nodes, embed_model=embed_model)
    return index, nodes

```

Chunks	Avg length	Min	Median	Max
204	1158.2 chars	174	1218	1530

A real chunk from this technique: the first one containing the \$200.00 late-fee fact (from `registration_and_attendance.txt`, 1275 characters, shortened here):

department that offers the course. Permission numbers become 'used' only after a successful enrollment request.

Adding Classes on the First Day of Instruction

At the first class meeting, Group 3 graduating students shall have top priority for any available spaces. An impartial procedure (such as a lottery or random selection) shall be used to choose, from among the graduating seniors, those who will fill available spaces. Automated waitlists run through the first nine days of the semester for most courses. Permission numbers will be required to add courses after that date (University Policy F20-1).

Adding Classes after the Last Day to Add: before the Census date, students can submit the Pre-census Late Enrollment

petition form to the Registrar's office (valid through census date per University Policy F09-2). After census date, students can petition for a late add using the Post-Census ...

2. Indexing (in memory)

The nodes are loaded into a `LlamaIndex VectorStoreIndex` with no external store configured, so `LlamaIndex` uses its default in-memory `SimpleVectorStore`. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

3. Retrieval-only output for the shared question (q3)

The retrieval function (documented once above) run on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
=== [token] q3: What are the two specialization tracks in SJSU's MS in Applied Data Intelligence ===
query embedding dim: 384, first 8 values: [-0.0274, -0.0687, -0.022, 0.0218, 0.0002, -0.0398, -0.0296, 0.0243]
query vector shape: (384,), stacked doc vectors shape: (5, 384)
rank store_score cosine_sim chunk_len source_file preview
1 0.675474 0.636141 1326 msda_curriculum_courses.txt use, and evaluate applied data
intelligence technologies and tools. Apply Necessary Skills Apply effective oral and written communication skills
necessary to pr
2 0.667953 0.644114 1434 applied_data_intelligence_ms_program.txt Program: Applied Data
Intelligence, MS - San José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=13784
Applied Data Intel
3 0.666494 0.653941 1362 data_science_ms_program.txt Program: Data Science, MS - San
José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=14075 Data Science, MS
Admissions Re
4 0.660618 0.636305 1005 msda_curriculum_courses.txt Program Frequently Asked
Questions Want to learn more about our MS in Applied Data Intelligence degree? Explore our FAQs page to get detailed
information about
5 0.656676 0.622502 1393 msda_curriculum_courses.txt data into actionable insights,
and step confidently into advanced roles across technology, business and research. Apply Now Request Information Why
Earn Your Ap
retrieval latency: 7.51 ms
```

Semantic chunking

1. Chunking

Embeds each sentence (`buffer_size=1`) with the same MiniLM model and starts a new chunk only where the embedding distance to the next sentence is in the top 5% (`breakpoint_percentile_threshold=95`). Chunks follow topic changes, so they are few and large, and their size varies with the text.

```
def build_semantic_index(docs, embed_model):
    splitter = SemanticSplitterNodeParser(
        buffer_size=SEMANTIC_BUFFER_SIZE,
        breakpoint_percentile_threshold=SEMANTIC_BREAKPOINT_PERCENTILE,
        embed_model=embed_model,
    )
    nodes = splitter.get_nodes_from_documents(docs)
    index = VectorStoreIndex(nodes, embed_model=embed_model)
    return index, nodes
```

Chunks	Avg length	Min	Median	Max
85	2434.5 chars	32	2133	7851

A real chunk from this technique: the first one containing the \$200.00 late-fee fact (from `registration_and_attendance.txt` , 5390 characters, shortened here):

Select the correct term. Choose the class you wish to add.

Two ways to select a class: if you know the Class Number, type the 5 digit Class Number and click Enter; or click the green "Search Button" (make sure the Course Career is set to Graduate or Undergraduate) and click "Select Class" to place it in your shopping cart.

Once you have chosen a class, a "Class Enrollment Options" page will appear. Verify the course, enter permission number if required, select any related component classes, and click okay. The Add Classes page will appear showing your classes in your shopping cart. You must click "Finish Enrolling" to actually add the class.

Component Classes: if the course has related components attached (e.g., lab, seminar), the system will prompt you to select the one you wish to add.

Department or Instructor Consent: classes designated "Department Consent Required" require a Perm ...

2. Indexing (in memory)

The nodes are loaded into a `LlamaIndex VectorStoreIndex` with no external store configured, so `LlamaIndex` uses its default in-memory `SimpleVectorStore`. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

3. Retrieval-only output for the shared question (q3)

The retrieval function (documented once above) run on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
=== [semantic] q3: What are the two specialization tracks in SJSU's MS in Applied Data Intelligence ===
query embedding dim: 384, first 8 values: [-0.0274, -0.0687, -0.022, 0.0218, 0.0002, -0.0398, -0.0296, 0.0243]
query vector shape: (384,), stacked doc vectors shape: (5, 384)
rank store_score cosine_sim chunk_len source_file preview
1 0.668653 0.6847 3777 msda_curriculum_courses.txt MS in Applied Data Intelligence |
Department of Applied Data Science Skip to main content Master of Science in Applied Data Intelligence Transform Data
Into Imp
2 0.667953 0.640085 2423 applied_data_intelligence_ms_program.txt Program: Applied Data
Intelligence, MS - San José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=13784
Applied Data Intel
3 0.659107 0.655045 1249 data_science_ms_program.txt Program: Data Science, MS - San
José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=14075 Data Science, MS
Admissions Re
4 0.656215 0.637509 3421 msda_curriculum_courses.txt Entertainment Waymo World Bank
Learn more about what alumni from SJSU's MS in Applied Data Intelligence - formerly the MS in Data Analytics - have
accomplished.
5 0.626916 0.602787 2397 data_analytics_ms_program.txt Program: Data Analytics, MS - San
José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=13&poid=7675 Data Analytics, MS
Admissions
retrieval latency: 5.38 ms
```

Sentence-window chunking

1. Chunking

Makes every sentence its own node and stores the three sentences on each side in the node's 'window' metadata (`window_size=3`). Only the single sentence is embedded and searched; the window is kept so surrounding context could be supplied later. Nodes are very small and numerous.

```
def build_sentence_window_index(docs, embed_model):
    splitter = SentenceWindowNodeParser.from_defaults(
        window_size=SENTENCE_WINDOW_SIZE,
        window_metadata_key="window",
        original_text_metadata_key="original_text",
    )
    nodes = splitter.get_nodes_from_documents(docs)
```

```
index = VectorStoreIndex(nodes, embed_model=embed_model)
return index, nodes
```

Chunks	Avg length	Min	Median	Max
1307	158.3 chars	3	124	4408

A real chunk from this technique: the first one containing the \$200.00 late-fee fact (from registration_and_attendance.txt , 241 characters):

Late Fee Assessed

Students registering for classes after census are assessed a late registration fee of \$200.00, which begins after census for any classes added (includes Retroactive Adds, which begin the first day after instruction ends).

Its window metadata (the 3 sentences either side, stored but not embedded):

Permission numbers will be required to add courses after that date (University Policy F20-1).

Adding Classes after the Last Day to Add: before the Census date, students can submit the Pre-census Late Enrollment petition form to the Registrar's office (valid through census date per University Policy F09-2). After census date, students can petition for a late add using the Post-Census Late Enrollment form.

Late Fee Assessed

Students registering for classes after census are assessed a late registration fee of \$200.00, which begins after census for any classes added (includes Retroactive Adds, which begin the first day after instruction ends). Students in the Step to College programs are not required to pay the late registration fee.

Waitlisting

SJSU has "waitlisting" for all classes. Waitlisting puts you in line to get a space when a class is full (closed class), should a space become available.

2. Indexing (in memory)

The nodes are loaded into a LlamaIndex `VectorStoreIndex` with no external store configured, so LlamaIndex uses its default in-memory `SimpleVectorStore`. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

3. Retrieval-only output for the shared question (q3)

The retrieval function (documented once above) run on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
=== [sentence_window] q3: What are the two specialization tracks in SJSU's MS in Applied Data Intelligence ===
query embedding dim: 384, first 8 values: [-0.0274, -0.0687, -0.022, 0.0218, 0.0002, -0.0398, -0.0296, 0.0243]
query vector shape: (384,), stacked doc vectors shape: (5, 384)
rank store_score cosine_sim chunk_len source_file preview
1 0.724425 0.645132 161 msda_curriculum_courses.txt Entertainment Waymo World Bank
Learn more about what alumni from SJSU's MS in Applied Data Intelligence - formerly the MS in Data Analytics - have
accomplished.
2 0.724354 0.645212 154 msda_curriculum_courses.txt SJSU's Master of Science in
Applied Data Intelligence elevates your technical proficiency and professional leadership, preparing you to drive
innovation.
3 0.712977 0.695679 124 msda_curriculum_courses.txt Apply to SJSU MSADI Program
Frequently Asked Questions Want to learn more about our MS in Applied Data Intelligence degree?
4 0.704737 0.609269 146 msda_curriculum_courses.txt Financial Aid Embrace Your Data-
Driven Future at SJSU Earn your Master of Science in Applied Data Intelligence and make real-world change happen.
5 0.690282 0.707374 551 applied_data_intelligence_ms_program.txt Program: Applied Data
Intelligence, MS - San José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=13784
Applied Data Intel
retrieval latency: 16.55 ms
```

4. Comparison

Retrieval quality (5 questions, k = 5)

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@k cosine	Recall@k	Mean retrieval latency (ms)
Token	204	1158.2 chars	0.7004	0.6412	0.80	7.91
Semantic	85	2434.5 chars	0.666	0.6011	0.80	5.25
Sentence-window	1307	158.3 chars	0.7646	0.6883	0.80	15.83

Top-1 cosine is the highest cosine among the top-5 chunks, averaged over the questions. Recall@k is file-level: a hit means the expected source file appears anywhere in the top 5. Latency covers the similarity search only and varies by machine.

Answer-level check (was the answer text itself retrieved?)

Recall@k only checks the file, so this supplementary check (`scripts/answer_level_check.py`) asks whether any top-5 chunk contains the expected answer text. Each cell is the rank of the first chunk that does; "-" means none did.

Technique	q1	q2	q3	q4	q5	Answered in top 5
Token	1	3	-	1	1	4/5
Semantic	2	3	-	-	1	3/5
Sentence-window	-	-	-	1	1	2/5

q1 requires two marker strings in one chunk, which disadvantages single-sentence nodes; q2, q4 and q5 use a single marker.

A confidently scored retrieval that does not contain the answer

For **q3** (the two specialization tracks of the MS in Applied Data Intelligence), **no technique** retrieved a chunk containing the answer. Under Sentence-window the top result has the highest store score of the whole run (0.7244): *"Entertainment Waymo World Bank Learn more about what alumni from SJSU's MS in Applied Data Intelligence - formerly the MS in Data Analytics - have accomplished."* It is an alumni-employer fragment with no specialization or elective in it. The program's own page does appear (rank 5) but only as its opening text, not the section that lists the specializations.

The embedding most likely rated it similar because it repeats the query's proper noun, "MS in Applied Data Intelligence". A single short sentence has little other context, so that name dominates the match. The marketing page repeats the program name often, and its text was not fully cleaned when the corpus was collected.

5. Observations

Sentence-window has the highest cosine scores (top-1 0.7646, mean@k 0.6883) but retrieved a chunk containing the answer for only 2 of 5 questions, against 4 for Token and 3 for Semantic. High similarity did not mean a useful chunk: single-sentence nodes carry little context, so shared proper nouns dominate the match, and near-identical boilerplate repeated across the three program pages (for example the GVAR sentence)

outranked the sentence naming DATA 270 in q2. Semantic chunks are the largest (2434.5 characters on average), which likely dilutes a single fact such as the \$200.00 fee in q4 inside a long registration page.

Recall@k is 0.80 for all three because of q1: the 3.0 GPA rule appears in three documents, and every technique retrieved `graduate_policies_procedures.txt`, a valid source that the single expected-file label counted as a miss. q3 failed for every technique. With only five questions these differences are indicative, not conclusive.

6. Conclusion

Token chunking (256 tokens, 32 overlap) is the best choice for this corpus. It retrieved a chunk containing the answer for 4 of 5 questions, against 3 for Semantic and 2 for Sentence-window, even though Sentence-window had the highest cosine scores, so cosine alone was misleading here. Its mean latency of 7.91 ms is well below Sentence-window's 15.83 ms and close to Semantic's 5.25 ms. The result rests on five questions, and q3 failed for all three techniques.